Spec: Tile Content by Station Category

Status: Draft **Date:** 2026-02-23

1. Problem Statement

Currently, all tiles in the scheduling grid display the same content regardless of which station category they belong to:

```
[CompletionIcon] {job.reference} · {job.client} · {element.name}
```

This is insufficient for operators: the information relevant to scheduling decisions varies significantly by station category. An offset press operator needs to see paper type and inking, while a folding machine operator needs format and pagination.

2. Goal

Introduce two **display modes** for tiles — **Produit** and **Tirage** — toggled via keyboard shortcut, allowing operators to switch between job identification and contextual production information at a glance.

3. Display Modes

3.1 Mode Produit (default)

The current behavior. All tiles display:

```
[CompletionIcon] {reference} · {client} · {element.name}
```

Example: `○ 24185 · Acme Corp · couv`

This mode answers: **"What job is this?"**

3.2 Mode Tirage (new)

Tiles display job ID, element name, then **station-category-specific production information** on a single line:

```
[CompletionIcon] {reference} · {element.name} • {category-specific content}
```

Example: `○ 24185 · couv • Couché Satin 115g 64x90 Q/Q 5000ex`

The `·` separates the job ID from the element name. The `•` separates identification from production info.

This mode answers: **"What are the production characteristics of this task?"**

The category-specific content varies by station category (see section 4).

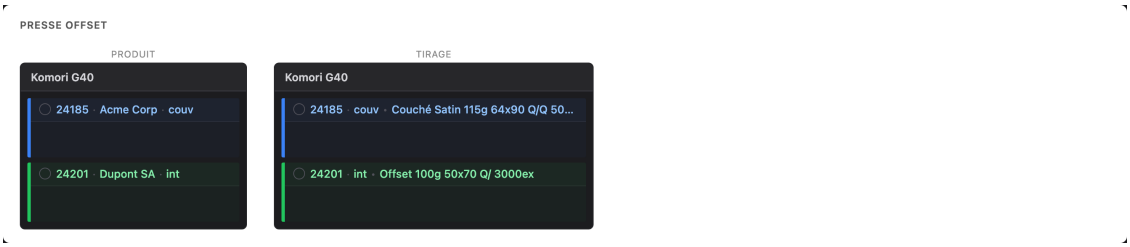
3.3 Toggle Behavior

PROPERTY	VALUE
Default mode	Produit
Keyboard shortcut	`A` (for "Affichage")
Condition	Only fires when no input/textarea is focused
Persistence	Session only (resets to Produit on page reload)
Visual indicator	Small text label in the header area showing current mode (e.g., `Affichage: Produit` or `Affichage: Tirage`)
Transition	Instant swap, no animation needed

3.4 Visual Preview

> *See companion file: `tile-preview.html` for interactive preview.*

Presse Offset — Produit vs Tirage:



Presse Offset

Massicot — Produit vs Tirage:

MASSICOT

PRODUIT	TIRAGE
Polar 137	Polar 137
24185 · Acme Corp · couv	24185 · couv · A6 5000ex
24310 · Martin & Fils · ELT	24310 · ELT · A4 1500ex

Massicot

Plieuse — Produit vs Tirage (leaflet + brochure):

PLIEUSE (DÉPLIANT + BROCHURE)

PRODUIT	TIRAGE
MBO S	MBO S
24055 · LeBlanc · ELT	24055 · ELT · A5f couché satin 90g 5000ex
24063 · Rivoli Print · int	24063 · int · A4f 16p couché satin 115g 1500ex

Plieuse

Encarteuse-Piqueuse — Produit vs Tirage:

ENCARTEUSE-PIQUEUSE

PRODUIT	TIRAGE
Hohner	Hohner
24185 · Acme Corp · fin	24185 · fin · A4f 2x16p + couv
24112 · Graphipole · fin	24112 · fin · 240x320/240x160 3x16p + 8p + 4p + couv

Encarteuse-Piqueuse

Assembleuse-Piqueuse — Produit vs Tirage:

ASSEMBLEUSE-PIQUEUSE

PRODUIT	TIRAGE
Horizon ASS	Horizon ASS
24201 · Dupont SA · fin	24201 · fin · A4f 48p + couv
24033 · Médiaprint · fin	24033 · fin · A5f 32p

Assembleuse-Piqueuse

Assembleuse — Produit vs Tirage:

ASSEMBLEUSE

PRODUIT	TIRAGE
Horizon 60H	Horizon 60H
24089 · Eurobook · fin	24089 · fin · 4 feuillets 500...

Assembleuse

Typographie — Produit vs Tirage:

TYPOGRAPHIE

PRODUIT	TIRAGE
SBG	SBG
24185 · Acme Corp · couv	24185 · couv · 70x102(4) 3000F offset 100g
24201 · Dupont SA · int	24201 · int · 50x70(2) 5000F couché brillant 300g

Typographie

Pelliculeuse — Produit vs Tirage:

PELLICULEUSE

PRODUIT	TIRAGE
Semipack	Semipack
24185 · Acme Corp · couv	24185 · couv · mat/mat 50x70(2) 3000F couché satin 115g

Pelliculeuse

4. Tirage Mode Content by Station Category

In Tirage mode, each tile displays: `{reference} · {element.name} • {category-specific content}`
The category-specific content part is defined below for each category.

4.1 Presse Offset (`cat-offset`)

```
{papier} {grammage} {formatFeuille} {impression} {quantite}ex
```

Full tile example: `24185 · couv • Couché Satin 115g 64x90 Q/Q 5000ex`

FIELD	SOURCE	NOTES
`papier`	`element.spec.papier` (type part)	Paper type name (e.g., “Couché Satin”, “Offset”)
`grammage`	`element.spec.papier` (weight part)	Extracted from papier DSL (e.g., “115g”)
`formatFeuille`	`element.spec.imposition` (dimensions part)	Sheet format (e.g., “64x90”)
`impression`	`element.spec.impression`	Recto/verso printing spec (e.g., “Q/Q”)
`quantite`	`element.spec.quantite`	Total copies (e.g., “5000”)

4.2 Massicot (`cat-cutting`)

```
{formatFini} {quantite}ex
```

Full tile example: `24185 · couv • A6 5000ex`

FIELD	SOURCE	NOTES
`formatFini`	`element.spec.format`	Finished product format
`quantite`	`element.spec.quantite`	Total copies

4.3 Plieuse (`cat-folding`)

Content depends on whether the job is a **brochure** or a **leaflet** (see detection logic in section 5.1).

Leaflet (Depliant):

```
{formatFini} {papier} {grammage} {quantite}ex
```

Full tile example: `24055 · ELT • A5f couché satin 90g 5000ex`

Brochure:

```
{formatFini} {pagination}p {papier} {grammage} {quantite}ex
```

Full tile examples: - `24063 · int • A4f 16p couché satin 115g 1500ex` - `24071 · int • 240x320/240x160 16p offset 100g 5000ex` - `24088 · int • A5f 32p couché satin 3000ex`

FIELD	SOURCE	NOTES
`formatFini`	`element.spec.format`	Finished product format
`pagination`	`element.spec.pagination`	Page count (brochure only)
`papier`	`element.spec.papier` (type part)	Paper type
`grammage`	`element.spec.papier` (weight part)	Grammage
`quantite`	`element.spec.quantite`	Total copies

4.4 Encarteuse-Piqueuse (`cat-booklet`)

```
{formatFini} {cahiersSummary}
```

Full tile examples: - `24185 · fin • A4f 2x16p + couv` - `24201 · fin • 240x320/240x160 16p` - `24112 · fin • 240x320/240x160 3x16p + 8p + 4p + couv` - `24310 · fin • A5f 2x16p + couv`

This is the most complex format. It summarizes the **cahiers** (signatures) from the job’s elements.

FIELD	SOURCE	NOTES
`formatFini`	`element.spec.format` (from the cover element, or first element)	Product format
`cahiersSummary`	Computed from job’s elements	See section 5.2

4.5 Assembleuse-Piqueuse (`cat-saddle-stitch`)

```
{formatFini} {paginationTotale}p [+ couv]
```

Full tile examples: - `24201 · fin • A4f 48p + couv` - `24033 · fin • A5f 20p + couv` - `24088 · fin • A5f 32p`

FIELD	SOURCE	NOTES
`formatFini`	`element.spec.format`	Product format
`paginationTotale`	Sum of `pagination` across all interior elements	Total page count
`+ couv`	Appended if a cover element exists	

4.6 Assembleuse (`cat-assembly`)

```
{nbFeuillets} feuillets {quantite}ex
```

Full tile example: `24089 · fin • 4 feuillets 5000ex`

FIELD	SOURCE	NOTES
`nbFeuillets`	`element.spec.pagination / 4`	**Calculated**: page count divided by 4
`quantite`	`element.spec.quantite`	Total copies

4.7 Typographie (`cat-typo`)

```
{imposition} {qteFeuilles}F {papier} {grammage}
```

Full tile examples: - `24185 · couv • 70x102(4) 3000F offset 100g` - `24201 · int • 50x70(2) 5000F couché brillant 300g`

FIELD	SOURCE	NOTES
`imposition`	`element.spec.imposition`	Sheet format with poses (e.g., “70x102(4)”)
`qteFeuilles`	`element.spec.qteFeuilles`	Sheet count
`papier`	`element.spec.papier` (type part)	Paper type
`grammage`	`element.spec.papier` (weight part)	Grammage

4.8 Pelliculeuse (`cat-pelliculeuse`)

```
{surfacage} {imposition} {qteFeuilles}F {papier} {grammage}
```

Full tile example: `24185 · couv • mat/mat 50x70(2) 3000F couché satin 115g`

FIELD	SOURCE	NOTES
`surfacage`	`element.spec.surfacage`	Coating recto/verso (e.g., “mat/mat”, “brillant/”)
`imposition`	`element.spec.imposition`	Sheet format with poses
`qteFeuilles`	`element.spec.qteFeuilles`	Sheet count
`papier`	`element.spec.papier` (type part)	Paper type
`grammage`	`element.spec.papier` (weight part)	Grammage

4.9 Other Categories (unchanged)

Conditionnement (`cat-packaging`) and any future groups: in Tirage mode, fall back to Produit display (`reference · client · element.name`).

5. Business Logic

5.1 Brochure vs. Leaflet Detection (for Plieuse)

A folding task belongs to a **leaflet** when: - The job is **single-element**, OR - The element’s workflow consists of **press + folding only** (no downstream assembly/booklet tasks)

A folding task belongs to a **brochure** when: - The job is **multi-element**, AND - Other elements in the job have tasks on assembly stations (encarteuse-piqueuse or assembleuse-piqueuse)

Implementation approach: Look at the sibling elements in the same job. If any sibling element has tasks assigned to `cat-booklet` or `cat-saddle-stitch` stations, the folding element is part of a brochure.

5.2 Cahiers Summary Computation (for Encarteuse-Piqueuse)

Given a multi-element job going through the encarteuse-piqueuse:

1. Identify the **cover element** (element with label containing "couv" or being the element with `prerequisiteElementIds` referencing all others, or the element whose tasks don't include a press step — to be refined)
2. Collect all **interior elements** with their `spec.pagination`
3. Group consecutive identical paginations: `[16, 16, 16, 8, 4]` -> `3x16p + 8p + 4p`
4. If cover exists, append `+ couv`

5.3 Papier DSL Parsing

The `element.spec.papier` field uses a DSL format: `{type}:{grammage}` (e.g., `"Couché mat:135"`).

- **Type extraction:** everything before the colon
- **Grammage extraction:** number after the colon, displayed as `{value}g`
- If no colon, the entire string is the type and grammage may come from `job.paperWeight`

5.4 Quantity Source

The quantity (`quantite` / `ex`) comes from the job creation form (JCF) and is stored in `element.spec.quantite`.

5.5 Feuillets Calculation (for Assembleuse)

```
feuillets = Math.ceil(pagination / 4)
```

Where `pagination` comes from `element.spec.pagination`. A feuillet is a sheet folded once, yielding 4 pages.

6. Tooltip Enhancement

6.1 Current State

Only blocked tiles show a tooltip (after 2s hover) displaying prerequisite blocking info.

6.2 Proposed Tooltip

All tiles get a rich tooltip on hover (shorter delay, e.g., 500ms) displaying comprehensive info regardless of current display mode. The tooltip always shows **both** Produit and Tirage information:

SECTION	CONTENT
Header	`{job.reference} — {job.client}`
Description	`{job.description}`
Deadline	`Sortie atelier: {workshopExitDate}`
Element	`{element.label ?? element.name}`
Spec	All non-null `ElementSpec` fields (format, papier, pagination, imposition, impression, surfacage, quantite, qteFeuilles)
Prerequisites	Paper / BAT / Plates / Forme status (if not all "none")
Task	Setup: {setupMinutes}min / Run: {runMinutes}min
Schedule	`{scheduledStart} -> {scheduledEnd}`

The tooltip is a custom React component (not browser-native `title`), styled consistently with the app's dark theme, and positioned to avoid overflow.

Blocked tile tooltip should be merged into this enhanced tooltip (showing prerequisite warnings with appropriate styling).

7. Column Width by Station Category

7.1 Current State

All station columns have the same width: **240px** (normal) / **120px** (collapsed).

7.2 Proposed Change

Column width is **fixed per station category**, independent of the display mode. It does not change when toggling between Produit/Tirage — only tile content changes.

Each category is assigned the width needed to accommodate the longest content across both modes:

CATEGORY	WIDTH	RATIONALE
`cat-offset`	340px	Long Tirage content (papier + grammage + format + impression + qty)
`cat-pelliculeuse`	400px	Longest Tirage content (surfacage + imposition + sheets + papier + grammage)
`cat-typo`	340px	Medium-long Tirage content (imposition + sheets + papier + grammage)
`cat-booklet`	400px	Cahiers summary can be long
`cat-saddle-stitch`	280px	Medium Tirage content
`cat-cutting`	240px	Short Tirage content, Produit ~240px
`cat-folding`	340px	Medium-long Tirage (brochure variant includes pagination + papier)
`cat-assembly`	240px	Short Tirage content, Produit ~240px
`cat-packaging`	240px	Unchanged (falls back to Produit in Tirage mode)

Implementation: Add a `columnWidth` lookup by `categoryId`, used by `StationColumn` instead of the hardcoded `w-60` class.

8. Data Availability Analysis

8.1 Fields Already Available

FIELD	SOURCE	AVAILABLE TODAY?
`job.reference`	`Job.reference`	Yes
`job.client`	`Job.client`	Yes
`element.spec.format`	`ElementSpec.format`	Yes (if populated)
`element.spec.papier`	`ElementSpec.papier`	Yes (if populated)
`element.spec.pagination`	`ElementSpec.pagination`	Yes (if populated)
`element.spec.imposition`	`ElementSpec.imposition`	Yes (if populated)
`element.spec.impression`	`ElementSpec.impression`	Yes (if populated)
`element.spec.surfacage`	`ElementSpec.surfacage`	Yes (if populated)
`element.spec.quantite`	`ElementSpec.quantite`	Yes (if populated)
`element.spec.qteFeuilles`	`ElementSpec.qteFeuilles`	Yes (if populated)

8.2 Data Gaps

1. **Cover element detection:** No explicit flag to identify a cover element. Must be inferred from `element.name` / `element.label` (contains "couv") or from element structure.
2. **Station category ID at tile level:** Currently not passed to the `Tile` component. The `station.categoryId` must be threaded through from `SchedulingGrid` -> `StationColumn` -> `Tile`.

8.3 ElementSpec Population

This feature is only useful if `ElementSpec` fields are populated during job creation (JCF). The JCF already captures these fields — verify that they are reliably saved and loaded.

9. Implementation Approach

9.1 Architecture Overview

```
App (displayMode state + keydown listener)
  └── SchedulingGrid (receives displayMode)
        └── StationColumn (receives station.categoryId -> width, + displayMode for tiles)
              └── Tile (receives categoryId + displayMode + element)
                    ├── Mode Produit: {reference} · {client} · {element.name} (unchanged)
                    ├── Mode Tirage: {reference} · {element.name} • {category-specific} (NEW)
                    └── TileTooltip (rich tooltip, always full info — NEW/ENHANCED)
```

9.2 New Components / Utilities

COMPONENT/UTILITY	PURPOSE
`useDisplayMode()`	Hook: manages `'produit' | 'tirage'` state + keydown listener for `A`
`tileLabelResolver.ts`	Pure function: `(categoryId, job, element, allElements, tasks) -> string`
`parsePapierDSL(papier: string)`	Extracts `{ type, grammage }` from papier DSL
`computeCahiersSummary(elements)`	Builds cahier summary for encarteuse-piqueuse
`detectBrochureOrLeaflet(job, elements, tasks)`	Returns `'brochure' | 'leaflet'`
`getColumnWidth(categoryId)`	Returns pixel width for a station column (fixed per category)
`TileTooltip` (enhanced)	Rich tooltip component for all tiles

9.3 Props Changes

`Tile` component — new props:

```
interface TileProps {
  // ... existing props ...
  /** Current display mode */
  displayMode: 'produit' | 'tirage';
  /** Station category ID for contextual content */
  categoryId: string;
  /** Element data for spec access */
  element?: Element;
  /** All elements in the job (for cahiers/brochure detection) */
  jobElements?: Element[];
  /** All tasks (for brochure detection) */
  allTasks?: Task[];
}
```

`StationColumn` — new props:

```
interface StationColumnProps {
  // ... existing props ...
  /** Station category ID (from station.categoryId) */
  categoryId?: string;
  /** Current display mode (passed to tiles, does not affect column width) */
  displayMode?: 'produit' | 'tirage';
}
```

9.4 Implementation Steps

1. **Create `useDisplayMode` hook** — state + keydown listener (skip when input focused)
2. **Create `tileLabelResolver.ts`** — pure function with unit tests for each category format
3. **Create `parsePapierDSL.ts`** — parser with tests
4. **Create `computeCahiersSummary.ts`** — algorithm with tests
5. **Create `detectBrochureOrLeaflet.ts`** — detection logic with tests
6. **Create `getColumnWidth.ts`** — width lookup by categoryId (fixed, independent of display mode)
7. **Modify `Tile` component** — conditionally render Produit or Tirage content based on `displayMode`
8. **Modify `StationColumn`** — accept `categoryId` for fixed width, + `displayMode` to pass to tiles
9. **Modify `SchedulingGrid`** — pass `categoryId`, `displayMode`, and `element` data through
10. **Wire up `useDisplayMode`** — in parent App/page component, pass down to grid
11. **Add mode indicator** — small text label in header showing current mode
12. **Build `TileTooltip` component** — rich tooltip for all tiles
13. **Update fixtures** — ensure `ElementSpec` fields are populated in test data

9.5 Testing Strategy

- **Unit tests:** `tileLabelResolver`, `parsePapierDSL`, `computeCahiersSummary`, `detectBrochureOrLeaflet`, `getColumnWidth`, `useDisplayMode`
- **Component tests:** `Tile` renders correct content per mode and category
- **Integration tests:** toggle `A` switches all tiles simultaneously
- **Visual QA:** verify column widths, text truncation, tooltip positioning

10. Future Enhancements (v1+)

- **Machine format limits:** Parameterize max sheet size per station, auto-recalculate `qteFeuilles` when task moves between stations (e.g., 70x100 press -> 50x70 typo = sheets x2)
- **User-configurable tile content:** Let users choose which fields to display per category via settings
- **Compact mode:** Alternative tile layout for zoomed-out views showing only the most critical info
- **Persist display mode:** Save preference in localStorage or user settings